

AI-DRIVEN ENTERPRISE PROJECT INTELLIGENCE & RISK MANAGEMENT PLATFORM

An intelligent platform for document-grounded project analysis, risk identification, project health assessment, and project intelligence

Document Control

Attribute	Details
Project Title	AI-Driven Enterprise Project Intelligence & Risk Management Platform
Student Name	Pankaj Kumar
Program	Infosys Springboard Internship
Project Domain	Generative AI, Retrieval-Augmented Generation (RAG), and Multi-Agent Workflows
Batch Code	M-2-6 (Batch 2)
Repository URL	GitHub Repository

Abstract

Project information in modern software engineering is distributed across project proposals, requirements documents, meeting minutes, task spreadsheets, sprint logs, and progress reports. Although these documents contain vital insights regarding project deliverables, dependencies, deadlines, and emerging technical blockers, manually consolidating and analyzing this information is slow, error-prone, and difficult to maintain over time.

The **AI-Driven Enterprise Project Intelligence & Risk Management Platform** solves this problem by transforming scattered project documents into a unified, searchable, and queryable project intelligence layer. The platform couples high-fidelity document parsing with semantic vector search (Retrieval-Augmented Generation) and an orchestrated multi-agent workflow to automate project analysis.

The architecture comprises a modern React and Vite frontend paired with a Python and FastAPI backend. Uploaded documents in multiple formats (PDF, DOCX, XLSX, CSV, TXT, MD, PPTX) are parsed into structured Markdown using IBM Docling, chunked with semantic overlap, embedded into dense vector representations, and stored in dedicated, project-isolated Qdrant vector collections. Structured project metadata, analysis outputs, and conversation histories are persisted in MongoDB via Beanie Object-Document Models.

A LangGraph-orchestrated multi-agent pipeline coordinates specialized agents for project scope extraction, multi-category risk identification, deterministic four-pillar health scoring, document auditing, and artifact generation (Executive Summaries, User Stories with acceptance criteria, Risk Registers, and Sprint Plans). A conversational AI assistant enables

natural-language querying with source citations, ensuring responses remain grounded in actual project documentation.

Keywords: Project Intelligence, Risk Analysis, Retrieval-Augmented Generation, Large Language Models, Semantic Search, Multi-Agent Workflow, Project Health, Document Intelligence, FastAPI, React, Qdrant, MongoDB.

01. Introduction

1.1 Background

Software engineering and project management activities produce large volumes of documentation throughout the software development lifecycle. At project initiation, proposals and charters outline the problem domain, core objectives, and expected milestones. During analysis and design, software requirements specifications (SRS) and architecture documents define structural constraints and interface contracts. During development and execution, sprint update notes, meeting records, task spreadsheets, and progress logs capture decisions, blockers, and changing timelines.

Although these project artifacts contain critical operational knowledge, the information is typically scattered across incompatible file formats and shared storage folders. As a result, building a consolidated, real-time understanding of project health, scope clarity, and delivery risks becomes an arduous manual task.

Traditional document repositories function primarily as passive storage systems. They store and retrieve files by name or folder, but they do not actively synthesize the text into structured project intelligence. Consequently, team leads, project managers, and quality assurance engineers must manually review multiple files to verify whether requirements are clearly specified, whether unmitigated risks exist, or whether milestone schedules are at risk of slipping.

Advances in artificial intelligence, dense vector embeddings, and Large Language Models (LLMs) provide a practical solution to this problem. Retrieval-Augmented Generation (RAG) enables language models to retrieve factual passages from a private document collection at query time before formulating answers. This approach is well suited for enterprise project intelligence because project documentation is unique to each team, evolves continuously, and requires verifiable answers that reference actual source documentation.

The **AI Project Intelligence & Risk Advisor** applies these principles to create an active intelligence layer over project artifacts. By combining document parsing, vector indexing, multi-agent AI analysis, deterministic mathematical scoring, and a modern web interface, the platform converts passive documentation into actionable project intelligence.

1.2 Problem Statement

Software development teams frequently experience planning gaps, undetected technical blockers, missed delivery deadlines, and incomplete requirement documentation. While teams continuously generate project artifacts, extracting actionable intelligence from these scattered files is time-consuming and often skipped due to daily delivery pressures.

The lack of a unified intelligence view leaves critical project signals disconnected:

- An unaddressed architectural dependency may be noted in a weekly meeting log.
- Incomplete functional requirements may be briefly mentioned in an email or chat summary.
- Looming deadline concerns may sit quietly inside a task spreadsheet.

When evaluated separately, these signals appear minor; however, when combined, they represent severe delivery risks. Without an automated platform to connect and evaluate these signals, teams remain trapped in reactive firefighting rather than proactive risk prevention.

The objective of the AI Project Intelligence & Risk Advisor is to provide an intelligent system where uploaded project files are automatically transformed into a unified, queryable knowledge base. The platform extracts structured scope parameters, detects multidimensional risks, computes an explainable project health score, synthesizes essential documentation, and provides an evidence-backed conversational assistant.

1.3 Motivation

The primary motivation of this project is to foster proactive risk awareness throughout the software development lifecycle.

Project documentation is produced continuously over time. Initial proposals set high-level goals, while subsequent sprint notes and meeting minutes document changes, technical debt, and team resource bottlenecks. Manually connecting these evolving data points requires repeated reading and cross-referencing.

An automated project intelligence platform minimizes this manual burden. By combining semantic search with specialized AI agents, the platform gathers evidence across multiple files, extracts structured insights, and presents them in an easy-to-read dashboard.

The goal is not to replace project managers, but to equip teams with an intelligent diagnostic assistant that surfaces hidden risks early, verifies documentation completeness, and speeds up decision-making.

1.4 Objectives

The major objectives of the platform are:

1. **Automated Document Ingestion:** Develop an ingestion pipeline capable of parsing common file types (.pdf, .docx, .xlsx, .csv, .txt, .md, .pptx, images) into clean Markdown while preserving document structure and tables.
2. **Project-Isolated Vector Knowledge Base:** Build a Retrieval-Augmented Generation pipeline using dense vector embeddings and isolated Qdrant collections to guarantee data boundaries between projects.
3. **Automated Scope Extraction:** Implement a specialized Scope Agent to extract structured deliverables, objectives, milestones, stakeholders, and out-of-scope items directly from uploaded files.
4. **Multidimensional Risk Detection:** Build a Risk Agent to identify both explicit and implied risks, classifying them across six core categories with severity, probability, impact, mitigation strategies, and source quotes.
5. **Deterministic Project Health Scoring:** Design an objective 0–100 health scoring engine based on four quantifiable pillars: Scope Clarity, Documentation Completeness,

Risk Density, and Schedule Risk.

6. **Automated Document Generation:** Implement an agent to synthesize four canonical project artifacts (Executive Summary, User Stories with acceptance criteria, Risk Register, and Sprint Plan).
 7. **Incremental Document Auditing:** Provide conditional execution logic to detect missing document types and generate only those required, saving processing time and LLM tokens.
 8. **Document-Grounded Conversational Assistant:** Build an interactive chat advisor capable of answering project queries with verified source citations.
 9. **Modern Web Workspace:** Deliver an intuitive, responsive web application supporting project management, document inspection, interactive analysis monitoring, and one-click PDF report exports.
-

1.5 Scope

The platform encompasses the full document-to-intelligence pipeline:

- **Project Workspace Management:** Creation, listing, inspection, and deletion of isolated project workspaces.
- **Document Ingestion:** Validation, disk storage, parsing via IBM Docling, recursive text chunking, and dense vector indexing in Qdrant.
- **Multi-Agent Analytical Workflow:** Automated execution of Scope, Risk, Health, Document Audit, and Document Generation agents coordinated via LangGraph.
- **Generated Artifacts Supported:**
 - Executive Summary (High-level project synthesis)
 - User Stories (Agile stories formatted with title, persona, acceptance criteria, priority, and estimation)
 - Risk Register (Structured Markdown table with severity ratings and mitigation actions)
 - Sprint Plan (Iterative multi-sprint roadmap with deliverables and milestone goals)
- **Conversational Intelligence:** Grounded question-answering with rolling conversation memory and source file attribution.
- **Presentation & Export:** In-browser document viewing (Word, Excel, PDF, Markdown) and client-side PDF intelligence report export.

Scope Boundaries: The current implementation focuses on document-grounded risk detection and deterministic health scoring. It does not include standalone time-series predictive models or direct integrations with external ticketing tools (e.g., Jira API).

1.6 Key Expected Outcomes

The platform produces the following concrete outcomes:

- A unified, queryable vector knowledge base derived from uploaded project documentation.
- A structured scope model summarizing project objectives, deliverables, and boundaries.
- An evidence-based risk register detailing risk category, severity, probability, impact, mitigation, and source context.
- A reproducible 0–100 Project Health Score with component-wise percentage

breakdowns.

- Four automatically synthesized project artifacts stored in the project repository.
 - A conversational advisor capable of answering natural-language project queries while citing source documents.
 - A web dashboard consolidating document viewing, risk monitoring, report generation, and interactive chat.
-

1.7 Design Principles

The design of the system adheres to eight foundational principles:

- **Evidence Grounding:** All AI-generated analyses and conversational answers must be strictly derived from retrieved project passages.
 - **Project Isolation:** Vector collections and document storage paths are strictly scoped per project ID to prevent cross-project context mixing.
 - **Structured Outputs:** Key analytical outputs use strict Pydantic schemas rather than unstructured free-form text.
 - **Deterministic Computation:** Quantitative evaluations (such as project health scores) use deterministic formulas rather than relying on language model estimation.
 - **Persistent Intelligence:** Analytical reports, risk items, and synthesized artifacts are stored in MongoDB and Qdrant to eliminate redundant model calls.
 - **Modular Architecture:** Ingestion, embedding, agent nodes, database persistence, and user interfaces are decoupled into distinct layers.
 - **Asynchronous Execution:** Heavy parsing, vector indexing, and multi-agent runs execute as background tasks, keeping API endpoints responsive.
 - **Traceability:** Risk items retain the exact text quotes from source documents that triggered the risk identification.
-

02. Existing Approach and Proposed Solution

2.1 Existing Approach

In standard software project management, documentation is treated as static text:

- Team members search through folders or cloud drives manually.
- Requirements, sprint updates, and meeting notes are inspected individually.
- Risk registers are manually updated in spreadsheets, often falling out of date.
- Project managers manually assemble status reports and executive summaries.

Keyword-based search tools (like traditional desktop or cloud drive search) only find exact word matches. They fail when equivalent concepts are described using different phrasing (for example, "milestone delay", "schedule slippage", "postponed release date", or "blocked dependency").

2.2 Proposed Solution

The proposed platform introduces an automated intelligence layer between raw documentation and project stakeholders.

Uploaded documents are transformed into searchable semantic passages. When an analysis is initiated, specialized agents query the project vector store to extract scope details, identify risks, and calculate project health. The resulting insights are displayed on an interactive dashboard and indexed back into the vector store, allowing the conversational assistant to query both raw files and generated reports.

2.3 Why RAG is Used

Retrieval-Augmented Generation (RAG) is chosen over fine-tuning or zero-shot prompting for three primary reasons:

1. **Dynamic Knowledge Updates:** When a new project document is uploaded, it is immediately indexed into Qdrant without requiring costly model retraining.
 2. **Elimination of Hallucinations:** Grounding LLM prompts in retrieved document passages ensures factual accuracy.
 3. **Source Verifiability:** RAG enables the system to tag every answer with the exact document name and section used to construct the response.
-

2.4 Proposed Processing Stages

The end-to-end processing pipeline comprises thirteen stages:

1. Create or select a project workspace.
 2. Upload project documentation (PDF, Word, Excel, CSV, text, etc.).
 3. Save the file to disk and record metadata in MongoDB (pending status).
 4. Asynchronously convert the document into clean Markdown using IBM Docling.
 5. Split the text into overlapping chunks using recursive character splitting.
 6. Attach project ID, document ID, filename, and file type metadata to each chunk.
 7. Generate dense vector embeddings and index them in the project's Qdrant collection.
 8. Trigger the multi-agent LangGraph analysis workflow.
 9. Execute Scope and Risk agents to extract scope parameters and categorize risks.
 10. Calculate the deterministic Project Health Score across four pillars.
 11. Audit existing documents and generate missing canonical artifacts.
 12. Persist the complete report to MongoDB and sync generated document chunks into Qdrant.
 13. Enable interactive dashboard inspection, report downloading, and grounded conversational Q&A.
-

03. Requirements Analysis

3.1 Functional Requirements

ID	Area	Functional Requirement Description
----	------	------------------------------------

FR-01 Authentication	Provide secure access to application endpoints and protect workspace resources.
FR-02 Project Management	Allow users to create, list, inspect, and delete project workspaces.
FR-03 Document Management	Support file uploads, document listing, file serving, and document deletion.
FR-04 Document Processing	Convert heterogeneous document formats into layout-preserved Markdown.
FR-05 Semantic Indexing	Chunk text, generate vector embeddings, and store them in Qdrant collections.
FR-06 Scope Analysis	Extract project name, objectives, deliverables, timeline, stakeholders, and out-of-scope items.
FR-07 Risk Analysis	Detect risks and extract title, description, category, severity, probability, impact, mitigation, and source context.
FR-08 Health Assessment	Calculate an explainable 0–100 health score with a four-pillar breakdown.
FR-09 Document Audit	Detect which canonical document types already exist and identify missing types.
FR-10 Document Generation	Synthesize missing Executive Summaries, User Stories, Risk Registers, and Sprint Plans.
FR-11 Analysis Report	Persist complete analysis outputs to MongoDB and provide retrieval endpoints.
FR-12 Analysis Status	Expose live pipeline status, active node step, and progress counters for UI polling.
FR-13 Conversational Assistant	Answer natural-language project questions grounded in indexed project context.
FR-14 Source Awareness	Return source attribution badges indicating referenced document titles.
FR-15 Generated Data View	Display synthesized Agile documents directly within the project workspace.
FR-16 Document Viewing	Provide in-browser previewers for DOCX, XLSX, PDF, CSV, and text files.
FR-17 PDF Output	Enable client-side rendering and downloading of formatted PDF intelligence reports.

3.2 Non-Functional Requirements

ID	Category	Non-Functional Requirement Description
NFR-01 Security		Protect API keys, credentials, and endpoints using environment configurations and access controls.
NFR-02 Isolation		Maintain dedicated Qdrant collections per project to eliminate cross-project data leakage.
NFR-03 Groundedness		Restrict generative responses to retrieved evidence, suppressing ungrounded assumptions.
NFR-04 Reliability		Isolate document processing failures, record error logs on affected records, and maintain pipeline stability.

NVIDIA Embeddings / Gemini Embed	Dense 2048-dimensional vector embedding generation.
Qdrant Vector Database	High-speed vector similarity database with dedicated per-project collections.
Google Gemini (3.6 Flash / 3.5 Flash Lite)	Large language model reasoning for multi-agent analysis and conversational RAG.
LangGraph	Stateful, directed graph orchestration for multi-agent analysis pipelines.
React 19	Modern, declarative component-based frontend framework.
Vite 8	Fast frontend bundler and development tooling with Hot Module Replacement.
TypeScript	Type-safe frontend implementation and contract alignment with backend schemas.
Tailwind CSS v4	Utility-first styling framework supporting modern dark-mode aesthetics.
react-markdown & remark-gfm	Rendering formatted Markdown text, checklists, code blocks, and tables in the UI.
PapaParse & SheetJS (xlsx)	In-browser CSV and Excel spreadsheet parsing and tabular previewing.
docx-preview	In-browser rendering of Microsoft Word documents.
html2pdf.js	Client-side conversion of generated Markdown reports into styled PDF downloads.
Docker & Docker Compose	Multi-container development and deployment environment.

4.1 Backend Structure

The backend is organized into clean, decoupled subpackages:

- **app/agents:** LangGraph state definitions, individual agent nodes (Scope, Risk, Health, Document Generator, Chat), and graph orchestrators.
 - **app/config:** Database connection initializers for MongoDB (Beanie) and Qdrant vector store management.
 - **app/controllers:** Request handlers implementing business logic for project workspaces and document handling.
 - **app/models:** Beanie document models and Pydantic schemas (Project, DocumentRecord, AnalysisReport, ChatSession, User).
 - **app/rag:** IBM Docling loader, text processors, embedding wrappers, and background ingestion pipeline.
 - **app/routes:** FastAPI route definitions for project, document, analysis, and chat endpoints.
-

4.2 Frontend Structure

The frontend is organized under `webui/src/`:

- **api/**: Axios client configured with backend endpoints and error interceptors.
 - **components/**: Reusable UI elements, layout shells (Header, Sidebar), file uploaders, in-browser document previewers, and PDF exporters.
 - **pages/**: Top-level application views (ProjectList, CreateProject, ProjectDetail).
 - **pages/tabs/**: Workspace tab components (Overview, DocumentsTab, AnalysisTab, ReportTab, ChatTab).
 - **types/**: TypeScript interface definitions aligned with backend Pydantic models.
-

4.3 External Services

The platform integrates four external services:

1. **MongoDB Server**: Local or Atlas cloud instance for structured document persistence.
 2. **Qdrant Vector Database**: Local container or cloud cluster for vector indexing.
 3. **Google Generative AI API**: Cloud API endpoint for Gemini LLM reasoning.
 4. **NVIDIA AI Endpoints**: Cloud API endpoint for high-dimensional text embeddings.
-

05. System Architecture

5.1 Architectural Overview

The system is structured into four cooperating layers:

1. **Presentation Layer (React / Vite)**: Handles user interactions, file drag-and-drop, tabbed navigation, live progress monitoring, document previews, and chat interfaces.
 2. **Application & API Layer (FastAPI)**: Exposes RESTful endpoints, validates request payloads, manages CORS, and dispatches background tasks.
 3. **Intelligence & Agent Layer (LangGraph & RAG)**: Orchestrates document retrieval, multi-agent reasoning, health scoring, document synthesis, and conversational answers.
 4. **Persistence Layer (MongoDB & Qdrant)**: Stores structured records, analysis reports, chat sessions, raw files, and 2048-dimensional vector representations.
-

5.2 High-Level Architecture

The component relationships and data flows operate across four distinct functional tiers:

Tier / Component	Associated Technologies	Primary Data Flow & Responsibilities
User Interface	React 19, Vite, Tailwind CSS v4, Lucide Icons	User creates project, uploads files, triggers multi-agent analysis, reviews health dashboard, and converses with the assistant.
API & Routing	FastAPI, Pydantic v2, BackgroundTasks	Validates incoming payloads, manages project lifecycles, serves document binaries, and manages asynchronous tasks.

Document Processing & RAG	IBM Docling, RecursiveSplitter, NVIDIA Embeddings	Converts files to Markdown, splits text into overlapping chunks, generates dense vectors, and manages per-project Qdrant collections.
Multi-Agent Orchestration	LangGraph, Gemini 3.6 Flash / Flash Lite	Executes Scope, Risk, Health, and Document Synthesis agents through a shared PipelineState, updating analysis reports.
Data Persistence	MongoDB (Beanie ODM), Qdrant Vector Store	Persists relational project records, analysis reports, conversation sessions, and semantic vector embeddings.

5.3 Project Isolation

To prevent cross-project data leakage, the vector database provisions a dedicated collection for every project using the naming convention `project_{project_id}`.

All embedding lookups, agent retrievals, and chat searches are strictly bound to this project-specific collection. When a project is deleted, its Qdrant collection is dropped immediately, guaranteeing complete isolation and straightforward cleanup.

5.4 Analytical State

The `AnalysisReport` document model maintains the complete state of a project's intelligence analysis. It stores:

- Overall analysis status (pending, running, ready, failed).
 - Current pipeline step for live UI tracking.
 - Extracted scope object (objectives, deliverables, timeline, stakeholders, out-of-scope items, summary).
 - Structured risk entries list (title, category, severity, probability, impact, mitigation, source context).
 - Deterministic health score and four component percentage breakdowns.
 - Synthesized canonical documents (Executive Summary, User Stories, Risk Register, Sprint Plan).
 - Document audit status (lists of existing and missing document types).
 - Error messages and timestamps.
-

06. Document Ingestion and Knowledge Base Construction

6.1 Supported Document Processing

The ingestion pipeline converts diverse document formats into a unified representation suitable for semantic indexing. Using the LangChain integration with **IBM Docling**, the loader parses incoming files and exports clean, layout-aware Markdown representations.

Supported file formats include Adobe PDF (`.pdf`), Microsoft Word (`.docx`, `.doc`), Microsoft

Excel (.xlsx, .xls), Comma-Separated Values (.csv), Plain Text (.txt), Markdown (.md), PowerPoint (.pptx), and scanned images.

6.2 Storage-to-Processing Flow

Document ingestion follows an eight-stage sequence:

Step Number	Processing Stage	Technical Implementation Detail
1	Upload & Metadata Creation	FastAPI receives file, verifies MIME type, saves to uploads/{project_id}/, and creates a DocumentRecord in MongoDB with pending status.
2	Background Ingestion Launch	FastAPI dispatches run_ingestion_pipeline as an asynchronous background task.
3	Status Transition	DocumentRecord status transitions to processing.
4	Layout-Aware Parsing	DocumentLoader invokes IBM Docling to convert the file into Markdown while preserving tables and headers.
5	Recursive Text Chunking	TextProcessor segments text into 1000-character chunks with a 400-character overlap using natural separators.
6	Metadata Enrichment	Every chunk is tagged with project_id, document_id, filename, and file_type.
7	Vector Embedding & Storage	Embeddings are computed and upserted into the project's Qdrant collection (project_{project_id}).
8	Completion & Chunk Count Update	DocumentRecord is updated to completed status with total chunk count; project total file/chunk counters update.

6.3 Chunking Strategy

The TextProcessor class uses RecursiveCharacterTextSplitter:

- **Source Document Chunk Size:** 1000 characters.
- **Source Document Overlap:** 400 characters.
- **Generated Document Chunk Size:** 1000 characters with 200-character overlap.
- **Separator Priority:** Splits along double newlines (\n\n), single newlines (\n), spaces (), and characters (" ").

This strategy prevents splitting sentences across chunk boundaries, ensuring that contextual relationships and tabular rows remain semantically intact.

6.4 Failure Handling

If a document fails during parsing (e.g., due to file corruption or unsupported encoding):

1. The exception is caught and logged with stack traces.

2. The `DocumentRecord` status in MongoDB is set to `failed`.
 3. The specific error message is saved to `processing_error` for UI display.
 4. The background worker terminates cleanly without crashing the server.
-

07. Retrieval-Augmented Generation Pipeline

7.1 RAG Concept in the Project

Rather than feeding entire multi-megabyte project archives to an LLM, the RAG layer retrieves only the top semantically relevant passages for a given task. This ensures fast response times, minimizes token costs, and keeps answers grounded in factual project evidence.

7.2 Embedding Model

- **Model:** `nvidia/nemotron-3-embed-1b` (NVIDIA AI Endpoints) or `gemini-embedding-001` (Google Generative AI).
 - **Vector Size:** 2048 dimensions.
 - **Distance Function:** Cosine Distance (\$1 - \text{Cosine Similarity}\$).
-

7.3 Project-Specific Vector Collection

Vector collections are initialized dynamically upon the first document upload. Collections use HNSW indexing in Qdrant for sub-millisecond approximate nearest neighbor (ANN) retrieval.

7.4 Indexing Generated Analysis

The vector store indexes both original uploaded files and AI-generated analytical artifacts:

1. **Analysis Report Summary:** Stored as a compact vector summarizing scope, health, and key risks.
2. **Generated Documents:** Executive Summaries, User Stories, Risk Registers, and Sprint Plans are split into 1000-character chunks (200 overlap) and indexed with deterministic UUIDs.

This enables the conversational assistant to query both raw requirements and synthesized reports seamlessly.

7.5 Retrieval for Scope Analysis

The Scope Agent executes three targeted semantic queries:

1. "project objectives goals deliverables"
2. "project scope timeline milestones stakeholders"
3. "what is out of scope requirements"

Each query retrieves up to 6 chunks. Duplicate chunks are filtered out, and the top 15 unique chunks are passed to the Scope Agent prompt.

7.6 Retrieval for Risk Analysis

The Risk Agent executes five specialized queries:

1. "risks blockers challenges problems delays"
2. "incomplete missing unclear requirements"
3. "deadline milestone schedule overdue"
4. "resource constraint team capacity dependency"
5. "technical debt complexity integration issues"

Each query retrieves up to 5 chunks. After deduplication, the top 18 unique passages are injected into the Risk Agent prompt.

7.7 Retrieval for Conversational Assistance

The Chat Agent runs a broad similarity search ($k = 12$) across the project collection. Chunks are extracted alongside source titles, combined with the last 6 conversation messages, and passed to Gemini 3.5 Flash Lite for grounded answer synthesis.

08. Project Scope Extraction

8.1 Scope Agent Responsibility

The Scope Agent extracts high-level project parameters from retrieved document passages and structures them into a validated `ScopeOutput` schema:

Output Field	Field Type	Semantic Meaning
project_name	String / Null	Name of the project supported by documentation.
objectives	List[String]	Key project goals and business objectives.
deliverables	List[String]	Expected software components, modules, or milestones.
timeline	String / Null	Delivery schedule, sprints, or milestone target dates.
stakeholders	List[String]	Identified project teams, clients, and user groups.
out_of_scope	List[String]	Features explicitly excluded from the current phase.
summary	String	2–3 sentence executive summary of project scope.

8.2 Grounding Rules

- Extract only facts directly supported by uploaded documentation.
- Use `null` or empty lists for unmentioned parameters (never invent placeholder data).
- Keep extracted list items concise, factual, and descriptive.

8.3 Structured Output and Fallback

The agent uses temperature `$0.1$` and instructs Gemini to return a strict JSON payload. If JSON deserialization encounters syntax irregularities (such as surrounding Markdown code fences), the parser strips the fences automatically. If parsing fails entirely, a truncated plain-text fallback summary is preserved, preventing pipeline interruption.

09. Risk Detection and Risk Analysis

9.1 Risk Agent Responsibility

The Risk Agent identifies both explicit risks (directly mentioned in notes) and implicit risks (ambiguous requirements, aggressive timelines, unverified third-party APIs). It extracts 5 to 15 structured `RiskItem` entries:

Field Name	Description
<code>title</code>	Concise title of the identified risk.
<code>description</code>	Detailed explanation of the underlying cause.
<code>category</code>	Risk classification (technical, resource, schedule, scope, external, quality).
<code>severity</code>	Severity rating (low, medium, high, critical).
<code>probability</code>	Probability of occurrence (Low, Medium, High).
<code>impact</code>	Potential consequences if the risk materializes.
<code>mitigation</code>	Concrete preventive action to avoid or reduce the risk.
<code>source_context</code>	Exact quote or paraphrase from the document that triggered the risk.

9.2 Risk Categories

Risk Category	Classification Criteria
Technical	Complex architecture, database schema changes, technical debt, integration issues.
Resource	Team capacity shortages, single points of failure, missing specialist skills.
Schedule	Overdue tasks, tight sprint deadlines, unestimated backlog items.
Scope	Ambiguous specifications, scope creep, undefined boundary conditions.
External	Third-party vendor dependencies, regulatory compliance, client approval delays.
Quality	Missing unit tests, lack of automated CI/CD, poor documentation standards.

9.3 Severity Levels

Severity Rating	Operational Meaning
Low	Minor concern with minimal impact on delivery; monitor periodically.
Medium	Moderate concern requiring proactive tracking and planned mitigation.
High	Significant blocker with potential to delay milestones; immediate action required.
Critical	Severe emergency threatening total project delivery; top priority resolution required.

9.4 Structured Parsing

Outputs are validated against the `RiskItem` Pydantic model. Enumerations for category and severity enforce standard terminology across the platform.

10. Project Health Scoring

10.1 Purpose

The Project Health Score provides an objective 0–100 numerical health assessment. The score is computed using a **deterministic mathematical formula** rather than asking an LLM to guess a number, ensuring identical scores for identical project inputs.

10.2 Health Components

The health score combines four weighted dimensions:

Component Name	Description & Evaluation Basis	Weight
Scope Clarity	Evaluates completeness across 8 core scope parameters (name, objectives, deliverables, timeline, stakeholders, summary).	30%
Documentation Completeness	Evaluates presence of out-of-scope items, ≥ 3 objectives, ≥ 3 deliverables, and stakeholders (25% each).	20%
Risk Density	Penalizes score based on the proportion of High and Critical severity risks among all identified risks.	30%
Schedule Risk	Penalizes score based on High and Critical risks categorized specifically under the "schedule" category.	20%

10.3 Formula

$$\text{Health Score} = (\text{Scope Clarity} \times 0.30) + (\text{Documentation Completeness} \times 0.20) + (\text{Risk Density} \times 0.30) + (\text{Schedule Risk} \times 0.20)$$

The resulting value is rounded to one decimal place and clamped to the 0.0–100.0\$ range.

10.4 Schedule Risk Calculation

Schedule risk evaluates time-sensitive blockers:

- If no risks exist: $\text{Schedule Risk Score} = 100.0$.
 - Otherwise:
$$\text{Schedule Risk Score} = \max\left(0.0, 100.0 - \left(\frac{N_{\text{high/critical schedule risks}}}{\max(N_{\text{total risks}}, 1)}\right) \times 100\right)$$
-

10.5 Health Breakdown

The `HealthBreakdown` model stores individual percentage scores. The frontend uses these values to display status tiers:

- ≥ 70.0 : Healthy (Green)
 - $40.0 - 69.9$: Moderate Risk (Yellow)
 - < 40.0 : Critical Risk (Red)
-

11. Automated Project Document Generation

11.1 Document Generation Strategy

The platform synthesizes four canonical project artifacts from extracted scope and risk data:

Generated Document	Content & Structure
Executive Summary	400–600 word Markdown summary with Project Overview, Key Objectives, Deliverables, Timeline, Stakeholders, and Conclusion.
User Stories	8–15 Agile user stories with titles, As a... I want... So that... structure, Acceptance Criteria checklists, Priority, and Story Point estimates.
Risk Register	Formatted Markdown table followed by detailed descriptions, impact statements, and mitigation strategies for every identified risk.
Sprint Plan	3–5 sprint roadmap with sprint themes, goals, estimated task checklists, and deliverable commitments.

11.2 Document Audit

Before generating documents, `doc_audit_node` queries MongoDB to identify which of the four document types already exist in `AnalysisReport`. If missing types are found, only those are generated. If all four exist, `skip_gen_node` avoids redundant LLM calls.

11.3 Generated Document Retrieval

Synthesized documents are saved to MongoDB and chunked into Qdrant vectors. This allows users to query generated user stories or sprint roadmaps directly in the chat tab.

12. Conversational Project Intelligence Assistant

12.1 Purpose

The Chat Agent provides natural-language Q&A over all project assets, allowing stakeholders to ask questions such as *"What are our primary technical risks for Sprint 1?"* and receive grounded answers.

12.2 Grounding Workflow

1. User submits a query in the Chat Tab.
 2. System retrieves existing conversation history from MongoDB.
 3. Query is embedded into a vector; Qdrant retrieves the Top-12 most relevant passages across raw files and generated reports.
 4. Passages are tagged with human-readable source labels.
 5. Prompt is assembled: System Instructions + Retrieved Passages + Last 6 Conversation Turns + User Query.
 6. Gemini synthesizes a grounded answer.
 7. Answer and source labels are stored in MongoDB and returned to the UI.
-

12.3 Grounding Behaviour

System instructions require the assistant to:

- Answer using only the provided context.
 - Explicitly state when information is missing rather than guessing.
 - Highlight risks and blockers proactively.
 - Format complex explanations with clean Markdown headers and bullet points.
-

12.4 Source Labelling

Source labels are generated from chunk metadata:

- **Uploaded files:** `Filename.pdf` or `Document_Title.docx`
 - **Generated documents:** `User Stories (generated)` or `Risk Register (generated)`
 - **Analysis summary:** `Analysis Report Summary`
-

13. Project Workspace and Dashboard

13.1 Frontend Project Structure

The frontend organizes project intelligence into a tabbed layout:

Tab / View	Functionality & Features
Overview	Project summary cards, health score gauge, quick risk counts, and pipeline status badges.
Documents	Drag-and-drop file upload, ingestion status badges (pending, processing, completed, failed), and in-browser preview drawer.
Analysis	One-click analysis runner, live node step tracker, interactive risk cards, and four-pillar health breakdown bars.
Reports	Tabbed viewer for synthesized documents (Executive Summary, User Stories, Risk Register, Sprint Plan) with PDF download.
Chat	Conversational assistant with multi-turn memory, typing indicator, and clickable source attribution badges.

13.2 Project Status

The workspace tracks project lifecycle states: `created` $\rightarrow$ `uploading` $\rightarrow$ `indexing` $\rightarrow$ `analysis_pending` $\rightarrow$ `analysis_running` $\rightarrow$ `analysis_ready` $\rightarrow$ `completed` (or `failed`).

13.3 Overview Health Presentation

The Overview tab presents the health score as a large radial gauge, accompanied by file counters, chunk totals, and quick-action buttons to launch analysis or upload additional files.

13.4 Document Viewing and PDF Output

- **In-Browser Document Previews:** Uses `docx-preview` for Word files, `SheetJS` for Excel tables, `PapaParse` for CSVs, and native renderers for PDFs and Markdown.

- **Client-Side PDF Export:** `html2pdf.js` compiles Markdown reports into professionally formatted PDF downloads with custom headers and page breaks.
-

14. Data Model and Persistence

14.1 Core Analysis Models

Model Name	Persistence	Key Fields & Stored Attributes
Project	MongoDB (projects)	name, description, status, total_files, total_chunks, current_health_score, health_score_history, associated_document_ids.
DocumentRecord	MongoDB (documents)	project_id, filename, file_type, file_size, storage_path, processing_status, chunk_count, processing_error.
AnalysisReport	MongoDB (analysis_reports)	project_id, status, scope, risks, health_score, health_breakdown, generated_documents, existing_doc_types, missing_doc_types, pipeline_step.
ChatSession	MongoDB (chat_sessions)	project_id, messages (list of ChatMessage objects with role, content, timestamp, and sources).

14.2 Risk and Scope Schema

- **RiskCategory Enum:** technical, resource, schedule, scope, external, quality.
 - **RiskSeverity Enum:** low, medium, high, critical.
 - **ScopeOutput Schema:** Strongly typed container for project_name, objectives, deliverables, timeline, stakeholders, out_of_scope, and summary.
-

14.3 Analysis Lifecycle

Analysis reports track execution state transitions: pending $\rightarrow$ running (with pipeline_step updates) $\rightarrow$ ready (or failed).

14.4 Vector Data

Vector payloads in Qdrant store:

- project_id, document_id, filename, file_type.

- `source_kind` (uploaded_document, generated_document, analysis_report).
 - `artifact_type`, `chunk_index`, `total_chunks`, and `page_content`.
-

15. API and Backend Implementation

15.1 API Responsibilities

The FastAPI backend exposes modular routers under `/v1/api/`:

- Request validation using Pydantic v2.
 - File upload handling and storage routing.
 - Background task dispatching for non-blocking analysis.
 - JSON error formatting and HTTP status code mappings.
-

15.2 Analysis Endpoints

HTTP Method	Endpoint Route	Operational Purpose
POST	<code>/v1/api/analysis/{project_id}/run</code>	Triggers full LangGraph analysis pipeline in the background.
POST	<code>/v1/api/analysis/{project_id}/generate-missing</code>	Incrementally generates only missing canonical documents.
GET	<code>/v1/api/analysis/{project_id}/status</code>	Returns current pipeline status, step, and progress metrics.
GET	<code>/v1/api/analysis/{project_id}/doc-audit</code>	Audits present versus missing generated document types.
GET	<code>/v1/api/analysis/{project_id}/report</code>	Retrieves complete analysis report and risk items.
GET	<code>/v1/api/analysis/{project_id}/documents</code>	Retrieves list of synthesized project documents.

15.3 Conversation Endpoint

HTTP Method	Endpoint Route	Operational Purpose
POST	<code>/v1/api/chat/{project_id}/message</code>	Accepts user message, performs RAG retrieval, returns answer with sources.
GET	<code>/v1/api/chat/{project_id}/history</code>	Retrieves full conversation message history for a project.
DELETE	<code>/v1/api/chat/{project_id}/history</code>	Clears conversation history for a project.

15.4 Long-Running Analysis

Pipeline triggers return immediate 200 OK or 202 Accepted responses. The heavy analysis runs in the background while the frontend polls /status to display live step progress (scope_node \$→ risk_node \$→ health_node \$→ doc_gen_node \$→ save_node).

15.5 Backend Modularisation

Separation of concerns is maintained across routes, controllers, services, models, and agents. CRUD operations and heavy AI processing operate independently without code coupling.

16. Authentication and Security

16.1 Authentication

The architecture supports JSON Web Token (JWT) based authentication. Protected routes validate bearer tokens from the Authorization header or secure cookies before processing requests.

16.2 Configuration

Configuration Setting	Security & Operational Purpose
JWT_SECRET_KEY	Cryptographic secret used to sign and verify user tokens.
JWT_ALGORITHM	Hashing algorithm (e.g., HS256).
ACCESS_TOKEN_EXPIRE_MINUTES	Configured session token lifespan.
MAX_UPLOAD_SIZE_MB	File size limit to prevent denial-of-service upload attacks.
API_ORIGIN	Whitelist of allowed frontend domain origins for CORS.

16.3 Data Protection Considerations

- AI API keys (Google, NVIDIA) are stored strictly in environment variables and never logged or sent to the client.
 - Uploaded files are stored in isolated per-project subdirectories.
 - Deleting a project cascades deletion across MongoDB, disk files, and Qdrant vector collections.
-

16.4 AI Security Considerations

Project documents are treated as untrusted data inputs. System prompts instruct the LLM to treat document text strictly as reference evidence rather than executable system instructions, preventing prompt injection attacks.

17. End-to-End System Workflow

17.1 Complete Workflow

The complete end-to-end operation comprises twenty-four discrete steps:

1. User creates or opens a project workspace.
2. User uploads project documentation via drag-and-drop.
3. Backend records `DocumentRecord` in MongoDB with `pending` status.
4. Ingestion background worker reads file from disk.
5. IBM Docling parses layout, headings, and tables into Markdown.
6. `RecursiveCharacterTextSplitter` segments text into overlapping chunks.
7. Metadata (project ID, document ID, filename, file type) is attached.
8. Embeddings are generated and inserted into Qdrant collection `project_{project_id}`.
9. `DocumentRecord` status transitions to `completed`.
10. User clicks "Run Analysis" on the Analysis Tab.
11. Backend initializes `PipelineState` and invokes `LangGraph`.
12. `scope_node` retrieves scope passages and extracts structured scope data.
13. `risk_node` retrieves risk passages and categorizes 5–15 risks.
14. `health_node` computes deterministic four-pillar health score.
15. `doc_audit_node` checks existing versus missing generated document types.
16. Conditional router directs flow to `doc_gen_node` or `skip_gen_node`.
17. `doc_gen_node` generates missing documents (Executive Summary, User Stories, Risk Register, Sprint Plan).
18. `save_node` writes `AnalysisReport` to MongoDB.
19. `save_node` chunks and upserts generated documents into Qdrant.
20. Project status updates to `analysis_ready`.
21. Frontend polls status and displays health breakdown and risk cards.
22. User reviews generated reports and exports formatted PDF summaries.
23. User submits natural-language questions in the Chat Tab.
24. Chat Agent queries Qdrant ($k=12$) and returns grounded answers with source citations.

17.2 Pipeline State

The `LangGraph` pipeline shares a strongly typed `PipelineState`:

- `project_id`: Target project identifier.
- `scope`: Extracted `ScopeOutput` object.

- `risks`: List of `RiskItem` objects.
 - `health_score` & `health_breakdown`: Computed metrics.
 - `existing_doc_types` & `missing_doc_types`: Document audit lists.
 - `generated_documents`: Accumulated list of generated artifacts merged via custom reducers.
 - `step_log`: Sequential execution log list.
 - `error`: Diagnostic error messages.
-

17.3 Conditional Generation

LangGraph conditional edges evaluate `missing_doc_types`:

- If missing types exist $\rightarrow$ Route to `doc_gen_node`.
- If no types are missing $\rightarrow$ Route to `skip_gen_node`.

Both branches converge at `save_node` before reaching `END`.

18. Testing and Evaluation Strategy

18.1 Testing Approach

The testing framework evaluates both deterministic code correctness and probabilistic AI quality:

Testing Area	Validation Scope & Focus
Project Management	Workspace creation, retrieval, listing, and cascading deletion.
Document Upload	MIME type validation, file storage, and upload limits.
Ingestion Pipeline	Docling layout parsing, chunk boundaries, metadata tagging, and Qdrant insertion.
Failure Handling	Corrupt file detection, error logging, and status transitions.
Scope Extraction	Schema validation, null field handling, and JSON cleaning.
Risk Detection	Categorization correctness, severity classification, and mitigation relevance.
Health Scoring	Mathematical formula precision, weighting accuracy, and boundary testing.
Document Generation	Markdown formatting, section presence, and incremental generation logic.
Conversational Chat	Retrieval relevance, source attribution, and refusal when context is missing.
Frontend UI	Navigation, tab switching, document preview rendering, and PDF exports.

18.2 Deterministic Tests

- Health score formula verification across boundary conditions (e.g., zero risks, 100% critical risks).
 - Schema validation for `RiskCategory` and `RiskSeverity` enumerations.
 - Document audit logic tests verifying all combinations of existing and missing document types.
 - Text chunking tests verifying 1000-character limits and 400-character overlap integrity.
-

18.3 AI Evaluation

- **Scope Extraction Completeness:** Verifying all stated deliverables in SRS files are captured.
 - **Risk Evidence Attribution:** Verifying that source quotes in risk items match actual document sentences.
 - **Retrieval Precision & Recall:** Evaluating Top-K passage relevance against ground-truth benchmarks.
 - **Faithfulness:** Measuring the absence of ungrounded or hallucinated statements in chat responses.
-

18.4 Historical Evaluation Information

Initial development evaluations conducted on a sample three-document project benchmark confirmed that multi-query retrieval, deterministic health scoring, and incremental document generation functioned as intended. Current benchmark evaluations are run against live instances to verify continuous correctness.

19. Results and Discussion

19.1 Functional Result

The platform delivers an end-to-end intelligence workflow. Users can upload heterogeneous documents, trigger automated multi-agent analysis, inspect categorized risks and health metrics, review synthesized Agile artifacts, and interact with a grounded conversational assistant.

19.2 Analytical Result

Analytical outputs are strongly structured rather than returned as monolithic text blocks. Scope deliverables, categorized risk items, health percentage breakdowns, and Agile stories are stored as structured objects, allowing the frontend to render interactive cards, filterable tables, and progress gauges.

19.3 Efficiency Result

Task-specific multi-query retrieval passes only relevant passages to each agent rather than dumping entire document archives into the LLM context. Furthermore, the `/generate-missing` workflow generates only required artifacts, significantly reducing API latency and token consumption.

19.4 Reusability Result

Because synthesized documents are chunked and indexed back into Qdrant, generated user stories and risk registers become part of the knowledge base. Users can ask conversational questions about previously generated artifacts, making the platform a unified, cumulative project memory.

19.5 User Experience Result

The user interface separates information into dedicated views (Overview, Documents, Analysis, Reports, Chat) rather than presenting a single overwhelming report. Users can preview original files side-by-side with extracted risks and download professional PDF summaries with one click.

20. Limitations and Future Scope

20.1 Current Limitations

- **Document Quality Dependency:** Incomplete or contradictory source documents lead to incomplete scope and risk extractions.
 - **External Cloud API Dependency:** Relies on internet connectivity for Google Gemini and NVIDIA embedding endpoints.
 - **OCR Processing Overhead:** Heavy scanned PDFs require additional processing time during Docling layout parsing.
 - **Static Artifact Analysis:** The system analyzes static text documentation rather than live code repositories or real-time commit feeds.
-

20.2 Areas for Future Enhancement

- **Jira & GitHub Integration:** Bidirectional synchronization of generated user stories, sprint tasks, and risk registers into Jira boards and GitHub Issues.
- **Hybrid Semantic & Keyword Search:** Combining dense Qdrant vector retrieval with BM25 sparse keyword indexing for exact technical term matching.
- **Meeting Audio Transcription:** Integrating Whisper models to directly transcribe Zoom/Teams meeting recordings into meeting minutes.
- **Enterprise Role-Based Access Control (RBAC):** Implementing multi-tenant organization tiers, team permissions, and Single Sign-On (SSO).
- **Automated Re-ranking:** Adding cross-encoder re-ranking models to further boost

retrieval precision for complex queries.

20.3 Research and Engineering Opportunities

Future research could explore genuine predictive delivery forecasting by combining document-grounded risk indicators with historical sprint velocity, pull request completion times, and dependency graph modeling.

21. Conclusion

The **AI-Driven Enterprise Project Intelligence & Risk Management Platform** provides an integrated solution for converting unstructured, scattered project documentation into structured, actionable project intelligence.

By combining **React**, **FastAPI**, **IBM Docling**, **MongoDB (Beanie)**, **Qdrant vector search**, **LangGraph**, and **Google Gemini**, the system bridges the gap between passive document storage and active AI-assisted project management. The platform extracts scope deliverables, identifies multidimensional risks, computes explainable health scores, generates essential Agile documentation, and enables grounded conversational Q&A with verifiable source citations.

The resulting platform reduces the manual effort required to oversee complex software projects, helping engineering teams detect blockers earlier, maintain documentation consistency, and deliver projects with greater confidence.

22. References

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
2. FastAPI Documentation. *FastAPI Framework Documentation*. <https://fastapi.tiangolo.com/>
3. MongoDB Documentation. *MongoDB Database Documentation*. <https://www.mongodb.com/docs/>
4. Beanie Documentation. *Asynchronous Python ODM for MongoDB*. <https://beanie-odm.dev/>
5. Qdrant Documentation. *Vector Database & Semantic Search Engine*. <https://qdrant.tech/documentation/>
6. LangChain Documentation. *Components for Document Processing, Embeddings, and LLMs*. <https://python.langchain.com/>
7. LangGraph Documentation. *Graph-based Orchestration for Stateful AI Workflows*. <https://langchain-ai.github.io/langgraph/>
8. Docling Documentation. *IBM Docling Document Conversion & Layout Analysis*. <https://github.com/DS4SD/docling>
9. React Documentation. *React Frontend Development Documentation*. <https://react.dev/>

10. Vite Documentation. *Vite Build Tooling Documentation*. <https://vitejs.dev/>
 11. Pydantic Documentation. *Data Validation and Typed Python Models*. <https://docs.pydantic.dev/>
 12. PyJWT Documentation. *JSON Web Token Implementation and Validation*. <https://pyjwt.readthedocs.io/>
 13. NVIDIA AI Endpoints Documentation. *NVIDIA Embedding Model Integration*. <https://build.nvidia.com/>
 14. Google Generative AI Documentation. *Google Gemini Model API Documentation*. <https://ai.google.dev/docs>
-

Appendix A – API Summary

Area	Endpoint Pattern	HTTP Method	Operational Purpose
Project	/v1/api/project/create	POST	Create a new project workspace.
Project	/v1/api/project/list	GET	List all project workspaces.
Project	/v1/api/project/{project_id}	GET	Retrieve single project metadata.
Project	/v1/api/project/{project_id}	DELETE	Delete project workspace and associated vector collection.
Documents	/v1/api/document/{project_id}/upload	POST	Upload document and launch background ingestion.
Documents	/v1/api/document/{project_id}/list	GET	List all uploaded documents for a project.
Documents	/v1/api/document/{project_id}/status/{doc_id}	GET	Poll ingestion and chunking status for a document.
Documents	/v1/api/document/{project_id}/view/{filename}	GET	Stream or download original document binary.
Documents	/v1/api/document/{project_id}/delete/{doc_id}	DELETE	Delete document record and file from disk.
Analysis	/v1/api/analysis/{project_id}/run	POST	Start full multi-agent analysis workflow.
Analysis	/v1/api/analysis/{project_id}/generate-missing	POST	Generate only missing document types for an existing analysis.
Analysis	/v1/api/analysis/{project_id}/status	GET	Poll analysis progress and current pipeline step.
Analysis	/v1/api/analysis/{project_id}/doc-audit	GET	Inspect generated document completeness.
Analysis	/v1/api/analysis/{project_id}/report	GET	Retrieve full analysis report and risk items.

Analysis	/v1/api/analysis/ {project_id}/documents	GET	Retrieve synthesized project documents.
Chat	/v1/api/chat/ {project_id}/message	POST	Submit query and receive grounded answer with source citations.
Chat	/v1/api/chat/ {project_id}/history	GET	Retrieve stored conversation history.
Chat	/v1/api/chat/ {project_id}/history	DELETE	Clear conversation history for a project.

Appendix B – Configuration and Deployment

B.1 Backend Configuration

Environment Variable	Description & Operational Role	Default / Example Value
MONGODB_URI	MongoDB connection address	mongodb://localhost:27017
DB_NAME	Database name for application state	ai_intelligence_risk_advisor
QDRANT_URL	Qdrant vector database server URL	http://localhost:6333
GOOGLE_API_KEY	Credential for Google Gemini Generative AI services	AIzaSy...
NVIDIA_API_KEY	Credential for NVIDIA embedding models	nvapi-...
LLM_MODEL	Identifier of the default language model	gemini-3.6-flash
HF_TOKEN	Hugging Face token where required by model stack	hf_...

API_ORIGIN	Allowed frontend origin for CORS configuration	http://localhost:5173
MAX_UPLOAD_SIZE_MB	Maximum allowed file upload size	50
JWT_SECRET_KEY	Secret key used to sign and validate JWT tokens	your-secret-key-here
JWT_ALGORITHM	Algorithm used for token validation	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Access token lifespan in minutes	1440

B.2 Development Sequence

1. Install Python 3.10+ and Node.js v18+ prerequisites.
 2. Start MongoDB (localhost:27017) and Qdrant (localhost:6333) instances.
 3. Configure backend environment variables in backend/.env.
 4. Install backend dependencies via `pip install -r requirements.txt`.
 5. Start the FastAPI server via `python app/main.py` (accessible at `http://127.0.0.1:3000`).
 6. Configure frontend environment variables in webui/.env (`VITE_API_URL=http://127.0.0.1:3000/v1/api`).
 7. Install frontend dependencies via `npm install`.
 8. Start Vite development server via `npm run dev` (accessible at `http://localhost:5173`).
 9. Create a project workspace, upload documents, execute analysis, and explore intelligence reports.
-

B.3 Container Deployment

The repository includes Dockerfiles and a `docker-compose.yml` file to orchestrate the frontend, backend, MongoDB, and Qdrant services in a unified container network for local evaluation and production hosting.

Appendix C – Data Structures and Output Formats

C.1 Risk Object

A structured `RiskItem` object contains:

- `title` (`String`): Concise risk summary.
 - `description` (`String`): Detailed explanation.
 - `category` (`Enum`): `technical`, `resource`, `schedule`, `scope`, `external`, `quality`.
 - `severity` (`Enum`): `low`, `medium`, `high`, `critical`.
 - `probability` (`String`): `Low`, `Medium`, `High`.
 - `impact` (`String`): Anticipated consequence.
 - `mitigation` (`String`): Recommended prevention action.
 - `source_context` (`String`): Supporting quote or paraphrase from uploaded documents.
-

C.2 Scope Object

A structured `ScopeOutput` object contains:

- `project_name` (`String` / `Null`): Identified project title.
 - `objectives` (`List[String]`): Core project goals.
 - `deliverables` (`List[String]`): Key deliverables and software modules.
 - `timeline` (`String` / `Null`): Milestone or delivery schedule description.
 - `stakeholders` (`List[String]`): Identified stakeholder groups.
 - `out_of_scope` (`List[String]`): Explicitly excluded scope items.
 - `summary` (`String`): High-level executive scope overview.
-

C.3 Health Breakdown

A structured `HealthBreakdown` object contains:

- `scope_clarity_percent` (`Float`): Completeness of scope parameters (30% weight).
 - `documentation_completeness_percent` (`Float`): Completeness of required documentation sections (20% weight).
 - `risk_density_percent` (`Float`): Proportion of non-critical risks (30% weight).
 - `schedule_risk_percent` (`Float`): Proportion of schedule-related risks (20% weight).
-

C.4 Generated Document

A structured `GeneratedDocument` object contains:

- `title` (`String`): Document title (e.g., "Executive Summary").
- `doc_type` (`String`): Canonical type identifier (`executive_summary`, `user_stories`, `risk_register`, `sprint_plan`).
- `content` (`String`): Full formatted Markdown content.
- `created_at` (`Datetime`): UTC timestamp.

C.5 Analysis Report

A complete `AnalysisReport` document combines:

- `project_id`, `status`, `error_message`, `pipeline_step`.
 - `scope`, `risks`, `health_score`, `health_breakdown`.
 - `generated_documents`, `existing_doc_types`, `missing_doc_types`.
 - `raw_outputs`, `created_at`, `completed_at`.
-

Appendix D – Glossary

Term	Operational Definition in Project
AI	Artificial Intelligence; computational techniques used for document parsing, reasoning, and text synthesis.
Agent	A specialized software component responsible for a dedicated analytical task (e.g., Scope Agent, Risk Agent).
Chunk	A bounded segment of processed document text (1000 characters) used as an independent retrieval unit.
Cosine Similarity	Angular similarity metric between two dense vectors, used to evaluate semantic closeness.
Docling	IBM document conversion library that extracts layout geometry and tables into clean Markdown.
Embedding	A 2048-dimensional dense numerical vector representing the semantic meaning of a text passage.
Grounding	The constraint requiring AI responses to be backed by retrieved project documentation.
LangGraph	Directed graph orchestration framework for stateful, multi-agent AI execution loops.
MongoDB	Document database used to persist structured project records, reports, and conversation turns.
Qdrant	High-performance vector database used to store and search dense vector embeddings.
RAG	Retrieval-Augmented Generation; retrieving relevant facts from a vector store before generating an answer.
Risk Item	A structured entity defining an identified risk, its category, severity, impact, mitigation, and source quote.
StateGraph	A stateful computational graph in LangGraph where nodes process and mutate a shared state object.

Appendix E – Implementation Verification

Notes

This project report documents the implemented system architecture and features of the AI-Driven Enterprise Project Intelligence & Risk Management Platform.

Key implementation milestones verified in the codebase include:

- IBM Docling layout-aware document parsing across PDF, DOCX, XLSX, CSV, and text formats.
 - Project-isolated vector collections in Qdrant with 2048-dimensional dense embeddings.
 - LangGraph multi-agent pipeline executing Scope, Risk, Health, and Document Synthesis agents.
 - Four-pillar deterministic health scoring formula combining scope clarity, documentation completeness, risk density, and schedule risk.
 - Smart incremental document generation (/generate-missing) via document auditing.
 - Pure RAG conversational assistant with 6-turn rolling memory and source file attribution badges.
 - In-browser document previewers (Word, Excel, PDF) and client-side PDF intelligence report export.
-